Praticando 1

Para cada função, responda: Quantas chamadas? Qual tamanho? Qual trabalho local? Qual recorrência? Qual método? Qual complexidade?

```
def A(n):
    if n <= 1:
        return
    A(n-1)
```

```
def B(n):
    if n <= 1:
        return
    B(n//2)
```

```
def C(n):
    if n <= 1:
        return
    C(n//2)
    C(n//2)
```

```
def D(n):
    if n <= 1:
        return

    D(n//2)
    D(n//2)

    for i in range(n):
        print(i)
```

Praticando 1A - GABARITO

```
def A(n):
    if n <= 1:
        return
    A(n-1)
```

Recorrência: $\boldsymbol{T(n) = T(n-1) + c}$

Caso base: $\boldsymbol{T(1) = C_0}$

Método: iteração.

Após $\boldsymbol{k}$ expansões: $\boldsymbol{T(n) = T(n-\mathrm{k}) + kc}$

Como a recursão termina quando $\boldsymbol{(n-\mathrm{k})}$, temos $\boldsymbol{(k = n-1)}$.

Portanto, $\boldsymbol{T(n) = T(1) + (n-1)c}$

Complexidade: $\boldsymbol{\theta(n)}$

Ver Exemplo 1 — Recursão linear

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 1 B - GABARITO

```
def B(n):
    if n <= 1:
        return
    B(n//2)
```

Recorrência: $T(n) = T\left(\frac{n}{2}\right) + c$

A cada chamada, o problema é reduzido à metade.

Método: iteração.

Após k expansões: $\frac{n}{2^k} = 1 \quad logo \;\; k = \log_2 n$

Complexidade: $\theta(log\, n)$

Ver Exemplo 3 — ÁRVORE DE RECURSÃO

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 1 C - GABARITO

```
def C(n):
    if n <= 1:
        return
    C(n//2)
    C(n//2)
```

Recorrência: $T(n) = 2T\left(\frac{n}{2}\right) + c$

Existem duas chamadas recursivas, cada uma com metade do tamanho do problema original. Pelo Método Mestre (veja detalhes na aula)

Como $n^{\log_2 2} = n$ e o trabalho local é constante, aplica-se o Caso 1

Complexidade: $\theta(n)$

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 1 D - GABARITO

```
def D(n):
    if n <= 1:
        return

    D(n//2)
    D(n//2)

    for i in range(n):
        print(i)
```

Recorrência: $T(n) = 2T\left(\frac{n}{2}\right) + cn$

O trabalho local é linear, pois o laço ***for*** executa n vezes.

Pelo Método Mestre $a = 2, b = 2$ e $f(n) = cn$

Como $n^{\log_2 2} = n$ aplica-se o Caso 2

Complexidade: $\theta(n \log n)$

Conclusão: o número de chamadas recursivas, isoladamente, não determina a complexidade. Também é necessário considerar o trabalho realizado em cada chamada.

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 2

a) Quantas chamadas recursivas existem?

b) Qual o tamanho do subproblema?

c) Qual o custo do trabalho local?

d) Construa a recorrência.

e) Resolva por iteração.

f) Determine Θ

```
def misterio(n):
    if n <= 1:
        return

    for i in range(n):
        print(i)

    misterio(n // 2)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 2 - GABARITO

A função ***misterio (n)*** executa um laço de ***n*** iterações e depois chama ***misterio (n//2)***

a) Quantas chamadas recursivas existem?

Uma chamada recursiva por execução.

b) Qual o tamanho do subproblema?

$n_{\text{subproblema}} = \lfloor n/2 \rfloor$

c) Qual o custo do trabalho local?

O laço executa ***n*** vezes,
portanto o trabalho local é $\boldsymbol{\theta(n)}$

```
def misterio(n):
    if n <= 1:
        return

    for i in range(n):
        print(i)

    misterio(n // 2)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 2 - GABARITO

d) Construa a recorrência.

Recorrência: $T(n) = T(n/2) + cn$ Caso base: $T(1) = c_0$

e) Resolva por iteração.

Expandindo: $T(n) = T(n/2) + cn$

$$T(n) = T(n/4) + c(n/2) + cn$$

$$T(n) = T(n/8) + c(n/4) + c(n/2) + cn$$

Após ***k*** expansões:

Praticando 2 - GABARITO

Expandindo: $\boldsymbol{T(n) = T(n/8) + c(n/4) + c(n/2) + cn}$

Após ***k*** expansões:

$$T(n) = T(n/2^k) + cn \sum_{i=0}^{k-1} \frac{1}{2^i}$$

A recursão termina quando $n/2^k = 1$, isto é, $k = \log_2 n$.

Como a soma geométrica é limitada por 2:

$$T(n) = T(1) + cn \sum_{i=0}^{\log_2 n - 1} \frac{1}{2^i}$$

Assim, o custo total é proporcional a n.

Praticando 2: gabarito

f) Determine Θ.

$$\boxed{T(n) = \Theta(n)}$$

O custo é linear, pois os trabalhos locais diminuem geometricamente: n, $n/2$, $n/4$, e assim por diante.

Esse exercício é muito interessante porque para mostra que, por haver recursão, não necessariamente n log n

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 3

a) Quantas chamadas recursivas existem?
b) Os subproblemas têm o mesmo tamanho?
c) A recorrência possui a forma do Método Mestre?
d) Escreva a recorrência.
e) Qual método você utilizaria? Justifique.

```
def processo(n):
    if n <= 1:
        return

    processo(n // 2)
    processo(n // 4)

    for i in range(n):
        print(i)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 3 - GABARITO

A função **processo(n)** chama **processo(n//2) e processo(n//4)**, além de executar um laço de **n** iterações.

a) Quantas chamadas recursivas existem?

Duas chamadas recursivas por execução.

b) Os subproblemas têm o mesmo tamanho?

Não. Os tamanhos são aproximadamente n/2 e n/4.

c) A recorrência possui a forma do Método Mestre?

Não diretamente. A forma clássica apresentada na aula $T(n) = aT(n/b) + f(n)$

```
def processo(n):
    if n <= 1:
        return

    processo(n // 2)
    processo(n // 4)

    for i in range(n):
        print(i)
```

Ela pressupõe subproblemas recursivos de mesmo tamanho. Neste caso, os tamanhos são diferentes.

Praticando 3 - GABARITO

d) Escreva a recorrência.

$T(n) = T(n/2) + T(n/4) + cn$

Com custo constante no caso base.

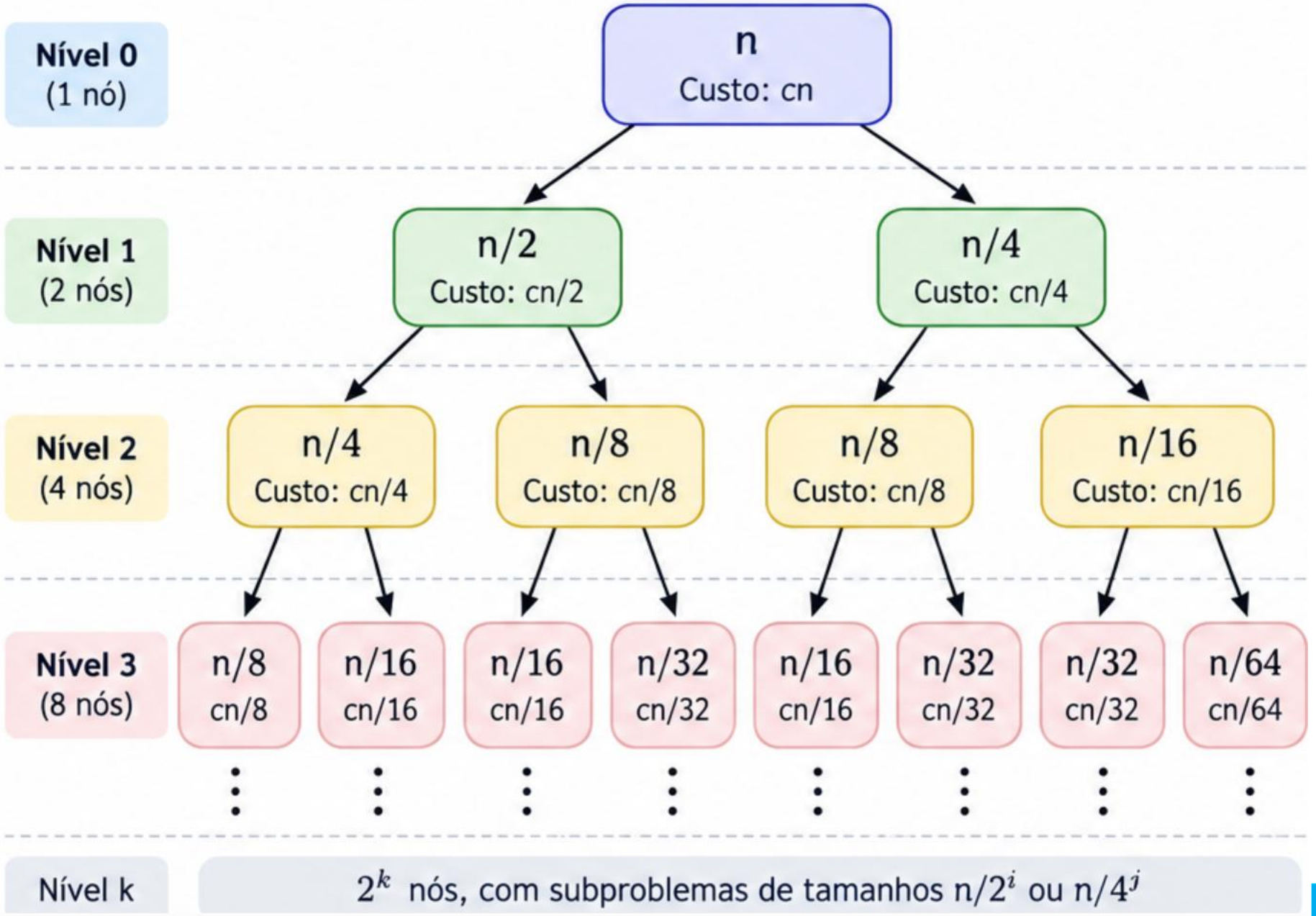
e) Qual método você utilizaria? Justifique.

Utilizaria a **árvore de recursão** ou a **expansão por recorrência**, porque os subproblemas têm tamanhos diferentes.

No nível inicial, o trabalho é proporcional a ***n***. No nível seguinte, a soma dos tamanhos dos subproblemas é aproximadamente:

$$\frac{n}{2} + \frac{n}{4} = \frac{3n}{4}$$

Praticando 3 - GABARITO



Praticando 3 - GABARITO

Nível	Tamanhos dos subproblemas (neste nível)	Custo total do nível
0	n	cn
1	$n/2,\ n/4$	$cn/2 + cn/4 = \frac{3}{4}cn$
2	$n/4,\ n/8,$ $n/8,\ n/16$	$\frac{cn}{4} + \frac{cn}{8} + \frac{cn}{8} + \frac{cn}{16}$ $= \frac{9}{16}cn$
3	n/8, n/16, n/16, n/32, n/16, n/32, n/32, n/64	$= \frac{27}{64}cn$
k	subproblemas de tamanho $n/2^i$ ou $n/4^j$	$\left(\frac{3}{4}\right)^k cn$

Praticando 3 - GABARITO

Custo total

$$T(n) = cn + \frac{3}{4}cn + \left(\frac{3}{4}\right)^2 cn + \left(\frac{3}{4}\right)^3 cn + \cdots = cn\left[1 + \frac{3}{4} + \left(\frac{3}{4}\right)^2 + \left(\frac{3}{4}\right)^3 + \cdots\right]$$

Como $0 < \frac{3}{4} < 1$, a soma da série geométrica é: $\sum_{k=0}^{\infty}\left(\frac{3}{4}\right)^k = \frac{1}{1-\frac{3}{4}} = 4$

Logo, $T(n) \leq 4cn + \mathcal{O}(1) = \mathcal{O}(n)$.

Como o nível 0 já custa $\Theta(n)$, temos $T(n) = \Omega(n)$.

Conclusão:

$$T(n) = \Theta(n)$$

Revisar série geométrica

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 4

Considere: $\boldsymbol{T(n) = 2T(n/2) + 1}$

a) Desenhe os quatro primeiros níveis.

b) Quantos nós existem no nível 0?

c) No nível 1?

d) No nível 2?

e) Qual o custo total de cada nível?

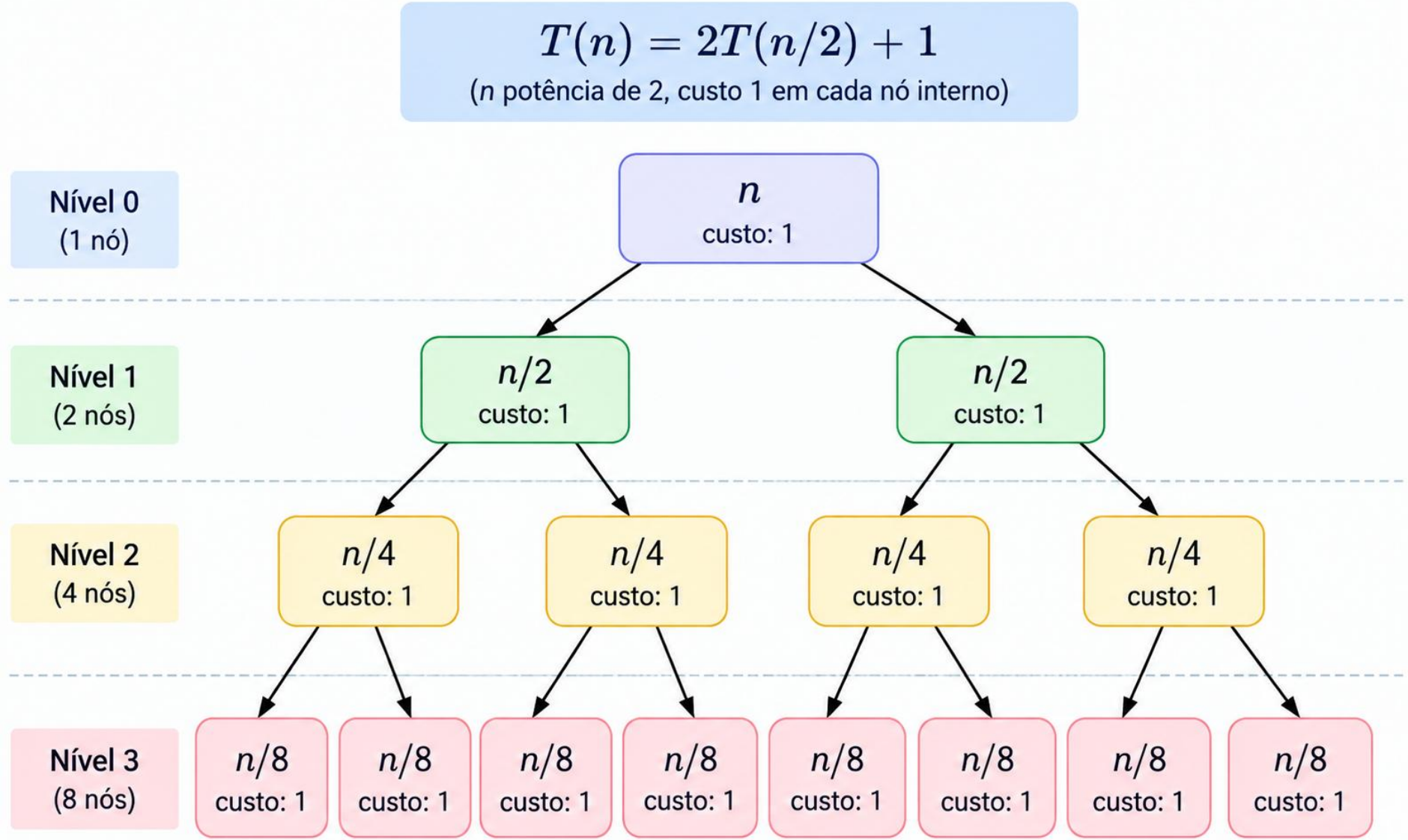
f) Quantos níveis existem?

g) Determine T(n).

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 4 - gabarito

a) Desenhe os quatro primeiros níveis.



Praticando 4 - gabarito

b), c) e d) Quantos nós existem em cada nível?

O número de nós dobra a cada nível.

Nível i	Número de nós	Custo total do nível
0	1	1
1	2	2
2	4	4
3	8	8
i	2^i	2^i

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 4 - gabarito

e) Qual o custo total de cada nível?

Como cada nó realiza trabalho constante, o custo do nível i é:

$$C_i = 2^i \cdot 1 = 2^i$$

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 4 - gabarito

f) Quantos níveis existem?

A recursão termina quando:

$$\frac{n}{2^k} = 1$$

Logo,

$$k = \log_2 n$$

Existem $\log_2 n$ divisões até o caso base, ou $\log_2 n + 1$ níveis se contarmos também o nível 0 e as folhas.

Praticando 4 - gabarito

g) Determine T(n).

Somando o custo dos níveis internos e o custo das folhas:

$$T(n) = 1 + 2 + 4 + \cdots + n$$

A soma é geométrica:

$$T(n) = \frac{2^{\log_2 n+1} - 1}{2 - 1}$$

Desconsiderando diferenças constantes de contagem das folhas, o custo é proporcional a n.

$$\boxed{T(n) = \Theta(n)}$$

Praticando 5

Algoritmo A: $\boldsymbol{T_A(n) = 2T_A(n/2) + n}$

Algoritmo B: $\boldsymbol{T_B(n) = 2T_B(n/2) + 1}$

a) Qual parece ser mais eficiente?
b) Qual é a complexidade de cada um?
c) Por que os dois possuem a mesma quantidade de chamadas recursivas, mas complexidades diferentes?
d) O que representa o termo f(n)?

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 5 - GABARITO

Algoritmo A: $\boldsymbol{T_A(n) = 2T_A(n/2) + n}$

Algoritmo B: $\boldsymbol{T_B(n) = 2T_B(n/2) + 1}$

a) Qual parece ser mais eficiente?

O algoritmo B, porque realiza trabalho constante fora da recursão em cada chamada, enquanto o algoritmo A realiza trabalho linear.

Praticando 5 - GABARITO

Algoritmo A: $\boldsymbol{T_A(n) = 2T_A(n/2) + n}$

Algoritmo B: $\boldsymbol{T_B(n) = 2T_B(n/2) + 1}$

b) Qual é a complexidade de cada um?

Para o algoritmo A:

$T_A(n) = 2T_A(n/2) + n$

Pelo Método Mestre, Caso 2:

$T_A(n) = \theta(n \log n)$

Para o algoritmo B:

$T_B(n) = 2T_B(n/2) + 1$

Pelo Método Mestre, Caso 1:

$T_B(n) = \theta(n)$

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 5 - GABARITO

c) Por que os dois possuem a mesma quantidade de chamadas recursivas, mas complexidades diferentes?

Ambos geram duas chamadas por execução, cada uma com metade do tamanho do problema. Entretanto, o trabalho local é diferente:

Algoritmo A: $\Theta(n)$ por chamada.
Algoritmo B: $\Theta(1)$ por chamada.

No algoritmo A, o custo de cada nível da árvore é proporcional a ***n***, produzindo $\Theta(n \log n)$
No algoritmo B, o custo total é dominado pela soma dos nós da árvore, que é $\Theta(n)$.

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 5 - GABARITO

d) O que representa o termo f(n)?

Representa o trabalho realizado na chamada atual, fora das chamadas recursivas.

Algoritmo A: $f(n) = n$ por chamada.
Algoritmo B: $f(n) = 1$ por chamada.

Praticando 6

a) Quantas chamadas recursivas?

b) Qual o custo do primeiro ***for***?

c) Qual o custo do segundo ***for***?

d) Qual o trabalho local total?

e) Construa a recorrência.

f) Resolva pelo Método Mestre.

```
def exemplo(n):
    if n <= 1:
        return

    exemplo(n // 2)

    for i in range(n):
        print(i)

    for j in range(n):
        print(j)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 6 - GABARITO

A função **exemplo(n)** realiza uma chamada recursiva com metade do problema e dois laços, cada um com ***n*** iterações.

a) Quantas chamadas recursivas?

Uma chamada recursiva: **exemplo(n//2)**

b) Qual o custo do primeiro ***for***?

$T_1(n) = \Theta(n)$

c) Qual o custo do segundo ***for***?

$T_2(n) = \Theta(n)$

d) Qual o trabalho local total?

Somando os custos:

$f(n) = cn + dn = \Theta(n)$

```
def exemplo(n):
    if n <= 1:
        return

    exemplo(n // 2)

    for i in range(n):
        print(i)

    for j in range(n):
        print(j)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 6 - GABARITO

e) Construa a recorrência.

$T(n) = T(n/2) + cn$

f) Resolva pelo Método Mestre.

Podemos identificar $a = 1, b = 2$ e $f(n) = \Theta(n)$

Assim: $n^{\log_2 1} = n^0 = 1$

```
def exemplo(n):
    if n <= 1:
        return

    exemplo(n // 2)

    for i in range(n):
        print(i)

    for j in range(n):
        print(j)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 6 - GABARITO

f) Resolva pelo Método Mestre.

Podemos identificar

$a = 1, b = 2 \;\; e \; f(n) = \Theta(n)$

Assim: $n^{\log_2 1} = n^0 = 1$

O trabalho local é polinomialmente maior que esse termo.

Além disso, a condição de regularidade é satisfeita para essa recorrência.

Portanto, pelo **Caso 3 do Método Mestre**:

$T(n) = \Theta(n)$

```
def exemplo(n):
    if n <= 1:
        return

    exemplo(n // 2)

    for i in range(n):
        print(i)

    for j in range(n):
        print(j)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 7

a) Quantas chamadas recursivas existem em cada chamada?
b) Quais são os tamanhos dos subproblemas?
c) Escreva uma recorrência aproximada para o número de chamadas.
d) Explique por que o Método Mestre não pode ser aplicado diretamente.
e) A árvore de recursão cresce rapidamente? Explique.

```
def fib(n):
    if n <= 1:
        return n


    return fib(n - 1) + fib(n - 2)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 7 - gabarito

a) Quantas chamadas recursivas existem em cada chamada?

Para n>1 existem duas chamadas recursivas.

b) Quais são os tamanhos dos subproblemas?

Os subproblemas possuem tamanhos ***n-1*** e ***n-2***

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 7 - gabarito

c) Escreva uma recorrência aproximada para o número de chamadas.

Se $C(n)$ representa o número de chamadas, podemos escrever:

$$C(n) = C(n-1) + C(n-2) + 1$$

Com $C(n) = C(n) = 1$, considerando também a chamada atual.

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 7 - gabarito

d) Explique por que o Método Mestre não pode ser aplicado diretamente.

Porque a recorrência não possui a forma $aT(n/b) + f(n)$**.**

Os subproblemas têm tamanhos diferentes: ***n-1*** e ***n-2***

Praticando 7 - gabarito

e) A árvore de recursão cresce rapidamente? Explique.

Sim. Muitas chamadas repetem o cálculo dos mesmos valores de Fibonacci. Por exemplo, o cálculo de `fib(n)` pode gerar diversas chamadas a `fib(n-2)`.

O número de chamadas cresce exponencialmente. A complexidade de tempo é:

$$\boxed{T(n) = \Theta(\varphi^n)}$$

em que $\varphi = (1 + \sqrt{5})/2 \approx 1{,}618$.

Também é comum apresentar o limite superior mais simples $O(2^n)$, que é correto, embora menos preciso.

Praticando 8

a) Qual a recorrência do tempo?

b) Qual a complexidade de tempo?

c) Quantas chamadas ficam simultaneamente na pilha?

d) Qual a complexidade de espaço?

```
def conta(n):
    if n <= 1:
        return

    conta(n - 1)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 8 - gabarito

a) Qual a recorrência do tempo?

Cada chamada realiza trabalho constante e chama a função com $n - 1$:

$$T(n) = T(n-1) + c$$

Com $T(1) = c_0$.

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 8 - gabarito

b) Qual a complexidade de tempo?

A recursão realiza aproximadamente n chamadas.

$$\boxed{T(n) = \Theta(n)}$$

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 8 - gabarito

c) Quantas chamadas ficam simultaneamente na pilha?

No pior momento, antes de começar a retornar, há aproximadamente n chamadas ativas, correspondentes a `conta(n)`, `conta(n-1)`, `conta(n-2)` e assim por diante.

Praticando 8 - gabarito

d) Qual a complexidade de espaço?

Cada chamada utiliza espaço constante na pilha. Como existem n chamadas simultâneas:

$$\boxed{S(n) = \Theta(n)}$$

Praticando 9

Compare o exercício 8 com esse:

```
def conta(n):
    if n <= 1:
        return

    conta(n // 2)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 9 - gabarito

a) Qual a recorrência do tempo?

A função realiza uma chamada recursiva com metade do tamanho e trabalho constante em cada execução:

$$T(n) = T(n/2) + c$$

Praticando 9 - gabarito

b) Qual a complexidade de tempo?

A cada chamada, o tamanho do problema é reduzido pela metade:

$$n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \ldots, 1$$

O número de chamadas é proporcional a $\log_2 n$.

$$\boxed{T(n) = \Theta(\log n)}$$

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 9 - gabarito

c) Quantas chamadas ficam simultaneamente na pilha?

Aproximadamente $\log_2 n$ chamadas.

d) Qual a complexidade de espaço?

Como cada chamada utiliza espaço constante:

$$\boxed{S(n) = \Theta(\log n)}$$

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 9 - gabarito

Comparação entre os praticandos 8 e 9

Característica	Praticando 8	Praticando 9
Redução do problema	$n - 1$	$n/2$
Tempo	$\Theta(n)$	$\Theta(\log n)$
Espaço na pilha	$\Theta(n)$	$\Theta(\log n)$

Conclusão: reduzir o problema pela metade a cada chamada diminui significativamente o número de chamadas recursivas, tanto em tempo quanto em espaço.

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 10

Considere: $T(n) = 2T(n/2) + n$

Um aluno afirma:
"Como existem duas chamadas recursivas e cada uma recebe metade do problema, então a complexidade é O(n)."

Você concorda? Explique.

Praticando 10 - gabarito

Não. A afirmação está incorreta.

O aluno considerou o tamanho dos subproblemas, mas não levou em conta o custo acumulado do trabalho realizado em todos os níveis da recursão.

Aplicando o Método Mestre:

- $a = 2$: duas chamadas recursivas.
- $b = 2$: cada subproblema tem tamanho $n/2$.
- $f(n) = n$: trabalho local linear.

Praticando 10 - gabarito

Calculamos:

$$n^{\log_b a} = n^{\log_2 2} = n$$

Como $f(n) = n$, estamos no Caso 2 do Método Mestre:

$$T(n) = \Theta(n^{\log_b a} \log n)$$

Portanto,

$$\boxed{T(n) = \Theta(n \log n)}$$

Praticando 10 - gabarito

$$\boxed{T(n) = \Theta(n \log n)}$$

A árvore de recursão ajuda a compreender o resultado: cada nível custa aproximadamente n, e existem $\Theta(\log n)$ níveis. Logo, o custo total é $\Theta(n \log n)$.

Resposta esperada: o aluno está errado porque o custo total depende tanto do número de chamadas quanto do trabalho realizado em cada nível da recursão.

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 11

Qual algoritmo é melhor?

$$A(n) = T(n-1) + 1$$

$$B(n) = T(n/2) + n$$

$$C(n) = 2T(n/2) + n$$

a) Determine a complexidade de cada uma.

b) Ordene os algoritmos do mais eficiente para o menos eficiente.

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 11 - gabarito

$$A(n) = \Theta(n)$$

$$B(n) = \Theta(n)$$

$$C(n) = \Theta(n \log n)$$

$$A \sim B < C$$

Ordem	Complexidade	Exemplo de comportamento
1	$\Theta(1)$	Custo constante
2	$\Theta(\log n)$	Redução pela metade
3	$\Theta(n)$	Percorrer os dados uma vez
4	$\Theta(n \log n)$	Divisão recursiva com trabalho linear por nível
5	$\Theta(n^2)$	Dois laços lineares aninhados
6	$\Theta(2^n)$	Recursão exponencial

A e B têm a mesma ordem assintótica, mas possuem estruturas recursivas completamente diferentes

Ibmec
PROTAGONISTAS PARA O MUNDO